

EE234L: Data Structures And Algorithms

Complex Engineering Activity

Library Management System

Weightage: 30 marks

Instructors: Ms. Bareera Sadia and Ms. Shehzeen Malik

Department of Electrical Engineering

University of Engineering and Technology, Lahore

1 Project Overview

Design and implement a library management system for UET that uses multiple data structures to efficiently manage books, members, and book lending operations. The system should have a **capacity 200 books** and **50 members**.

1.1 Problem Statement

The UET Library needs a system to:

- Manage book catalog with details (**ISBN, title, author, year, category, copies available**)
- Maintain member records (**ID, name, type: student, borrowed books**)
- Process book borrowing and returns
- Search books by ISBN, title, or author

2 Required Data Structures

2.1 Book Catalog (AVL Tree)

- **Key:** **ISBN** (13-digit number as string)
- **Value:** Book object with title, author, year, category, available_copies
- **Operations:** Insert, search, delete in **$O(\log n)$** time
- Maintain balance after all operations
- Support **inorder traversal for sorted ISBN listing**

2.2 Title Index (Hash Table)

- **Key:** Book title (normalized - lowercase, no extra spaces)
- **Value:** ISBN number
- **Operations:** **$O(1)$** average-case lookup
- Handle collisions using chaining with linked lists
- Support exact title matching

Design Note: When a user searches by title, use the hash table to find the ISBN, then use that ISBN to retrieve full book details from the AVL tree.

2.3 Author Index (Hash Table)

- **Key:** Author name (normalized)
- **Value:** Singly linked list of ISBNs
- **Operations:** $O(1)$ lookup for author
- **One author can have multiple books** (stored in linked list)

2.4 Member Database (Hash Table)

- **Key:** Member ID (e.g., “2024-EE-001”)
- **Value:** Member object with name, and list of borrowed ISBNs
- **Operations:** $O(1)$ lookup, insertion, update
- **Store borrowed books as a simple Python list** (limited to 5 books per member)

3 Required Functionality

3.1 Core Implementation)

1. AVL Tree for Books

- Implement Node class with ISBN, book data, left/right pointers, height
- Implement insert, search, and delete operations
- Implement rotation methods (rotate_left, rotate_right)
- Maintain balance factor and perform rotations as needed
- Implement inorder traversal to list all books sorted by ISBN

2. Hash Tables

- Implement hash table class with chaining for collision resolution
- Create title index: title $\rightarrow$ ISBN
- Create author index: author $\rightarrow$ list of ISBNs
- Create member database: member_id $\rightarrow$ member object
- Implement insert, search, and delete operations

3. Basic Operations

- Add book to system (insert into AVL tree and both indices)
- Add member to system
- Search book by ISBN (AVL tree)
- Search book by title (hash table $\rightarrow$ AVL tree)
- Search books by author (hash table $\rightarrow$ AVL tree for each ISBN)

3.2 Advanced Features

1. Borrowing System

- Implement borrow_book(member_id, isbn) method
- Check book availability (available_copies > 0)
- Check member's borrowing limit (max 5 books)
- Update book's available_copies
- Add ISBN to member's borrowed list

2. Return System

- Implement return_book(member_id, isbn) method
- Remove ISBN from member's borrowed list
- Increment book's available_copies

3. Optional: **Reporting Functions**

- List all books by a specific author
- List all books borrowed by a specific member
- List all available books (`available_copies > 0`)
- List all books in sorted order (by ISBN)

4 Implementation Guidelines

4.1 Data Structure Integration

The three data structures work together as follows:

1. **AVL Tree** stores complete book information indexed by ISBN
2. **Title Hash Table** provides quick title lookup, stores only ISBN
3. **Author Hash Table** provides author lookup, stores linked list of ISBNs

Example: To search by title:

```

1 # Step 1: Look up ISBN in title hash table (O(1))
2 isbn = title_hashtable.search(title)
3
4 # Step 2: Look up full book details in AVL tree (O(log n))
5 if isbn:
6     book = avl_tree.search(isbn)
7     return book

```

4.2 File I/O (Optional)

If you want to load data from files, you can use this approach:

books.csv format:

```

ISBN,Title,Author,Year,Category,TotalCopies
9780134685991,Effective Python,Brett Slatkin,2019,Programming,3
9780132350884,Clean Code,Robert Martin,2008,Programming,5

```

Reading hint:

```

1 import csv
2
3 with open('books.csv', 'r') as file:
4     reader = csv.DictReader(file)
5     for row in reader:
6         isbn = row['ISBN']
7         title = row['Title']
8         author = row['Author']
9         # Create book object and insert into AVL tree
10        # Insert title->ISBN mapping in title hash table
11        # Insert author->ISBN mapping in author hash table

```

5 Deliverables

1. Source Code

- Well-commented Python implementation
- Separate files for each data structure (`avl_tree.py`, `hash_table.py`, `library_system.py`)
- Clear class and method names

2. Test Suite

- Comprehensive tests for all operations
- Test with at least 50 books and 20 members
- Edge case testing (empty library, borrowing when unavailable, etc.)

3. Demo Video (5 minutes maximum)

- Demonstrate adding books and members
- Show search by ISBN, title, and author
- Demonstrate borrowing and returning books
- Show report generation (Optional)

5.1 Team Formation

- Groups of 2-3 students
- Individual contributions must be documented in README
- Use Git for version control (recommended but optional)

5.2 Code Quality Expectations

- Proper documentation
- Meaningful variable and function names
- Error handling for edge cases
- Efficient implementations matching theoretical complexity

6 Important Notes

- Focus on correct implementation of data structures first
- File I/O is optional - you can **hardcode test data**
- Ensure AVL tree maintains balance (**verify with tests**)
- Hash table should **handle collisions properly**
- All search operations must use the appropriate data structure (don't iterate through everything)
- The title and author hash tables store only ISBNs, not full book objects

6.1 Academic Integrity

- All code must be your own work
- You may discuss concepts but not share code between teams
- Cite any external resources used
- Plagiarism results in zero marks for all team members

Good luck with your project!